Exercise 1 — Knowing Where to Start

Exercise Description

Practise using AI prompts to understand the **Task Manager** codebase as if it were an unfamiliar project you've just encountered. Apply three prompt scenarios to build understanding of the project structure, locate specific features, and understand the domain model, then plan a new business rule.

Codebase used: `ai-code-exercises/use-cases/code-algorithms/python/TaskManager` (Python implementation).

Part 1 — Understanding Project Structure

Initial assumptions (before deep reading): a small CLI task manager, file-based persistence, no framework.

Findings after exploration (validated):

File	Responsibility
`models.py`	Domain layer — `Task` entity plus `TaskPriority` (LOW/MEDIUM/HIGH/URGENT enum, values 1-4) and `TaskStatus` (TODO/IN_PROGRESS/REVIEW/DONE enum). Contains task behaviour: `update()`, `mark_as_done()`, `is_overdue()`.
`storage.py`	Persistence layer — `TaskStorage` reads/writes `tasks.json`; custom `TaskEncoder` / `TaskDecoder` handle enum + `datetime` (de)serialisation. In-memory dict keyed by task id.
`task_manager.py`	Application/service layer — `TaskManager` orchestrates CRUD, filtering, tag management, and statistics over `TaskStorage`.
`cli.py`	Presentation layer — `argparse` command surface (`create`, `list`, `status`, `priority`, `due`, `tag`, `untag`, `show`, `delete`, `stats`) plus `format_task()` for display.
`task_priority.py`	Algorithm — weighted scoring (`calculate_task_score`, `sort_tasks_by_importance`, `get_top_priority_tasks`).
`task_parser.py`	Algorithm — natural-language parsing (`parse_task_from_text`) of `@tag`, `!priority`, `#date` markers.
`task_list_merge.py`	Algorithm — two-way sync/merge with conflict resolution.

- **Technology stack:** pure Python 3 standard library — `argparse`, `json`, `enum`, `uuid`, `datetime`, `re`, `copy`. No third-party dependencies, no database, no web framework.
- **Architecture:** a clean layered design — CLI → TaskManager (service) → TaskStorage (persistence) → Task/enums (domain). The three algorithm modules are standalone and operate on `Task` objects.

- **Entry points:** `cli.py:main()` (real CLI entry, guarded by `if __name__ == "__main__"`). `task_manager.py` also imports `argparse` but its `main` is effectively unused — a small dead-code smell.

Corrected misconception: I assumed persistence might be sprinkled through the manager; in fact it is cleanly isolated in `TaskStorage`, and `TaskManager` never touches JSON directly. Every mutating operation calls `storage.save()`.

Questions I'd ask the team: Why does `task_manager.py` carry its own `argparse` import/`main` when `cli.py` is the entry point? Is `tasks.json` the intended long-term store or a placeholder for a DB? Are the three algorithm modules wired into the CLI, or are they library code for future features?

Part 2 — Finding Feature Implementation (Task Export to CSV)

Search approach: grepped for `csv`, `export`, `write`, `open(`. There is currently **no** export or CSV functionality; the only file I/O is JSON persistence in `storage.py`.

Where the feature belongs (implementation plan):

1. **Service method** — add `export_to_csv(self, path, status_filter=None)` to `TaskManager`. It should reuse `list_tasks()` for selection so filtering behaviour stays consistent.
2. **Serialisation** — write with the stdlib `csv` module (`csv.DictWriter`). Columns: `id, title, description, status, priority, due_date, tags, created_at, completed_at`. Flatten enums to `.value`/`.name` and `datetime` to ISO strings (mirroring the logic already in `TaskEncoder`).
3. **CLI wiring** — add an `export` subparser in `cli.py` (`task_id` not needed; args: output path, optional `--status`).
4. **No changes needed** to `models.py` or `storage.py` — export is a read-only projection.

Components affected: `task_manager.py` (new method), `cli.py` (new command). Reusable existing logic: the enum/datetime flattening in `TaskEncoder`.

Part 3 — Understanding the Domain Model

Core entities:

- **`Task`** — the aggregate root. Identity is a UUID string. Fields: `title`, `description`, `priority`, `status`, `created_at`, `updated_at`, `due_date`, `completed_at`, `tags` (list of strings).
- **`TaskPriority`** — enum LOW(1) < MEDIUM(2) < HIGH(3) < URGENT(4). Numeric values give an ordering used by filtering and (with different weights) by scoring.
- **`TaskStatus`** — enum lifecycle: TODO → IN_PROGRESS → REVIEW → DONE. String-valued for readable JSON.

Relationships & rules (in business terms): A task moves through a status lifecycle; reaching DONE stamps `completed_at` (via `mark_as_done()`), which the statistics use for "completed in last 7 days". "Overdue" is a derived property (`is_overdue()`): a due date in the past **and** status not DONE. Tags are free-form labels used for filtering and to boost priority score (`blocker` / `critical` / `urgent`).

Domain glossary: *Task, Priority, Status, Overdue (derived), Due date, Completed-at (set only on DONE), Tag, Score (computed importance, not stored).*

Diagram (text):

```
TaskManager ──uses──> TaskStorage ──holds──> {id: Task}
Task ──has──> TaskStatus (enum)   Task ──has──> TaskPriority (enum)   Task ──has──> [tags]
Task.is_overdue() ← derived from due_date + status
```

Part 4 — Practical Application

New business rule: *Tasks overdue for more than 7 days are automatically marked as "abandoned" unless they are high priority.*

Plan:

1. **Domain change (`models.py`):** add `ABANDONED = "abandoned"` to `TaskStatus`. Add a method `days_overdue()` returning `(now - due_date).days` when overdue else `0`. "High priority" = `priority in (TaskPriority.HIGH, TaskPriority.URGENT)`.
2. **Service change (`task_manager.py`):** add `auto_abandon_stale_tasks()` that iterates tasks and sets status to `ABANDONED` where `days_overdue() > 7` and priority is not HIGH/URGENT, then saves once.
3. **Where to trigger it:** call on `list` / `stats` (lazy sweep) or expose a dedicated CLI command; a scheduled/cron trigger would be ideal in production.
4. **Ripple effects to check:** `format_task()` status symbols, statistics `by_status` counts (add "abandoned"), and `is_overdue()` (an abandoned task should arguably no longer count as "overdue").

Questions for the team before implementing: Is "more than 7 days" strictly `> 7` or `>= 7`? Should ABANDONED be reversible? Should abandoning be automatic on read, or an explicit maintenance job?

Reflection: The prompts were most useful for Part 2 — they turned "where does export go?" into a concrete, layered plan by forcing me to state the existing architecture first. Remaining uncertainty: the intended runtime wiring of the three algorithm modules. Next step: trace an end-to-end `create → list → done` flow through the CLI to confirm the service/storage contract.

Submission for Exercise 1 — Knowing where to start

1. Initial vs. final understanding of the Task Manager codebase: Initially assumed a small CLI with mixed concerns; final understanding is a cleanly **layered** architecture (CLI → service → storage → domain) with three standalone algorithm modules and zero third-party dependencies.

2. Documented findings per part: structure table + entry points (Part 1); CSV-export belongs as a read-only `TaskManager.export_to_csv` + CLI command reusing `list_tasks` and the encoder's flattening logic (Part 2); domain model = `Task` aggregate with `TaskPriority`/`TaskStatus` enums, derived `is_overdue`, and `completed_at` set only on DONE (Part 3).

3. Plan for the new business rule: add `ABANDONED` status + `days_overdue()` to the domain, an `auto_abandon_stale_tasks()` sweep in the service (skip HIGH/URGENT), trigger via CLI/scheduled job, and update display + statistics.

4. Most valuable insight from each prompt: structure prompt → the strict layering; feature-location prompt → reuse existing selection/serialisation rather than adding a new path; domain prompt → "overdue" and "score" are *derived*, not stored, which is exactly where the new rule must hook in.

5. Strategies for unfamiliar code in future: read outside-in (entry point → service → domain), state your hypothesis before asking AI so you can catch your own misconceptions, and map features to layers before writing any code.